AI Capstone

Homework 4: A Robot Manipulation Framework

Announcement: 4/28, Deadline: 5/12 23:59

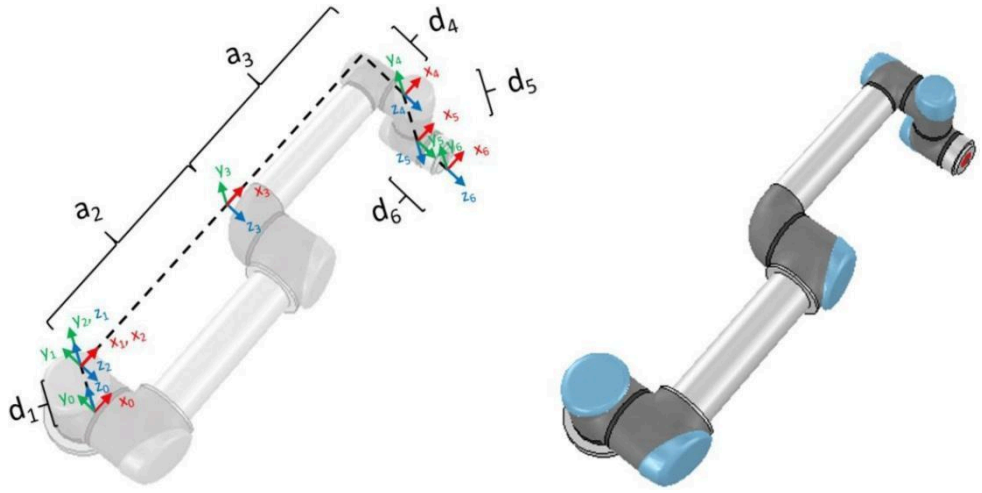
Introduction

In this homework, you are required to implement the forward kinematics (FK) and inverse kinematics (IK) functions for a 6-degree-of-freedom robot arm. Besides, you need to answer questions about the robot manipulation framework developed in this homework.

There are two main tasks:

- Implement the forward kinematics function.
- Implement the inverse kinematics function.

We will use the Nvidia Isaac Sim simulator for this homework. The robot arm used in this homework is ur5, which is a 6-degree-of-freedom robot arm (containing 6 revolute joints).



The 6-degree-of-freedom robot arm we will use in this homework

Requirements

The requirement of the development environments(or you can use Glows.AI and follow the slides):

- OS :
    - Ubuntu 22.04 (recommended)

      You can use uv to create a virtual environment, or use Docker to run the application in a containerized environment.
    - macOS

      Isaac sim only support nvidia GPU
    - Windows 10/11

      You can use uv to create a virtual environment, or use Docker to run the application in a containerized environment.
- Python 3.11

NVIDIA Isaac Sim System Requirements for Local Deployment

(Glows.AI will handle setup automatically in their environment)

The requirement of the Nvidia Isaac Sim simulator:

Requirement	Minimum	Recommended	Ideal
OS	Ubuntu 22.04 / 24.04 or Windows 10 / 11	N/A	N/A
CPU	Intel i7 (7th Gen) / AMD Ryzen 5	Intel i7 (9th Gen) /…	Intel i9 / AMD Ryz…
Cores	4	8	16
RAM	32GB	64GB+	N/A
Storage	50GB SSD	500GB SSD	1TB NVMe SSD

GPU	RTX 4080	RTX 5080	RTX PRO 6000 Bl…
VRAM	16GB	16GB+	48GB
Driver (Linux)	580.65.06 or newer	N/A	N/A
Driver (Windows)	580.88 or newer	N/A	N/A

Driver Recommendations

Linux Drivers I

- **R570 (Production Branch)**
  - **GeForce: 570.169**
  - **Workstation: 570.169**
- **R580 (Recommended, Production Branch)**
  - **GeForce: 580.95.05**
  - **Workstation: 580.95.05**
  - **Data Center: 580.95.05**
- **Supported Architectures: Ada / Ampere / Turing/ Blackwell**

Linux Drivers II

- **GeForce: 550.144.03**
- **Workstation: 550.144.03**
- **Data Center: 550.144.03**
- **Supported Architectures: Ada / Ampere / Turing**

Windows Drivers I

- **R570**
  - **GeForce: 581.42**
  - **Workstation: 573.42**

    - **Data Center: 581.42**
- **R580 (Recommended)**
    - **GeForce: 581.42**
    - **Workstation: 581.42**
    - **Data Center: 581.42**
- **Supported Architectures: Blackwell**

Windows Drivers II

- **R570**
    - **GeForce: 581.42**
    - **Workstation: 573.42**
    - **Data Center: 573.39**
- **R580 (Recommended)**
    - **GeForce: 581.42**
    - **Workstation: 581.42**
    - **Data Center: 581.42**
- **Supported Architectures: Ada / Ampere / Turing**

Important Notes

- Isaac Sim **container is Linux-only**
- Internet connection is required for assets and extensions
- More RAM/VRAM is needed for:
    - Large scenes
    - High-resolution rendering (>16MP)
    - Multi-sensor workloads

Compatibility Notes

- Older drivers are not supported
- Newer drivers may work but are not fully validated

Choose one way to create an environment. **If you don’t have the resources to**

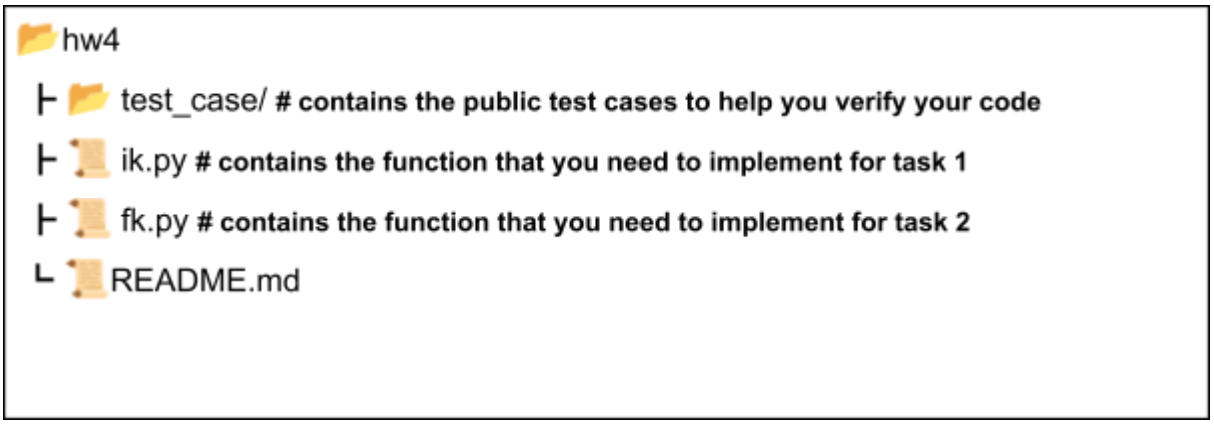
run the Nvidia Isaac Sim simulator, we encourage you to find your team members and classmates who have the resources to finish the homework 4.

Implementation

We will provide a project template of the robot manipulation framework. **Please complete this homework based on this template**.

For task 1 and task 2, you are required to implement the functions that we have defined in the project template and make them output the expected results.

The file structure of the project template:



Please read the following instructions in README.md **CAREFULLY**.
[Important] Please check all the **"TODO"** comments in **fk.py, ik.py**

Task 1: Forward kinematics function

The input and output of function **your_fk()** in **fk.py**:

- Input:
  - A set of joint states (the rotation angle of the robot arm) which is a 6D array ranging from -π to π
  - The D-H parameters (following **classic convention**) of the robot arm which is a dictionary structure. We have already provided it in **fk.py**.
- Output:
  - The corresponding 7D pose of the robot end-effector:
    - 3D position: x, y, z in the world coordinate
    - 4D rotation: quaternion in (x, y, z, w) format
  - The corresponding Jacobian matrix in $R^{6 \times 6}$

You need to modify **your_fk()** function in **fk.py** and make it output the expected results. **You cannot use any Nvidia Isaac Sim or third-party functions to directly output the results. You will not get any points in this part if you do not follow the policy**. Of course, the libraries about mathematical computing or matrix operations are allowed to use (e.g. numpy, quaternion, numba, scipy.spatial.transform …).

To verify the correctness of your implementation, you can use a subset of testing cases to check the corresponding scores. We will use the same scoring function to verify your results (import **your_fk()** in our scoring scripts).

The D-H table in the official specification is slightly different in this homework. Please follow the D-H parameters we provided in **fk.py** or you may fail to implement this function.

Hint: Prior Knowledge

- D-H parameters for classic convention
- geometric Jacobian in robotics

Task 2: Inverse kinematics function implementation

The input and output of this function **your_ik()** in **ik.py**:

- Input:
    - Target 7D pose of the robot arm's end-effector
        - 3D position: x, y, z in the world coordinate
        - 4D rotation: quaternion in (x, y, z, w) format
    - Current 6D joint parameters ranging from -π to π
        - 6D: for the revolute joints
- Output:
    - The expected 6D joint parameters for the target 7D pose of the robot arm's end-effector
        - 6D: for the revolute joints (will be different from the input)

In this task, you are required to implement the **iterative inverse kinematics (IIK)** using the Jacobian method (please check this slide). You need to implement the **pseudo-inverse method** to compute the desired joint parameters for the target robot arm's end-effector pose. Of course, it is welcome to implement other IK methods you like, you can compare the results with the pseudo-inverse method and observe the differences between them. You may get some extra points if you also try other methods to implement the IK function.

Similar to part 1, You need to modify **your_ik()** function in **ik.py** and make it output the expected results. **You cannot use any Nvidia Isaac Sim or third-party functions to directly output the results. You will not get any points in this part if you do not follow the policy.** Besides, you can use a subset of testing cases to check the corresponding scores.

Hint: You may need to implement **your_fk()** function in this task.

References

- Introduction to the Isaac Sim: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html
- D-H parameters of UR5 robot: https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-fo r-calculations-of-kinematics-and-dynamics/
- Inverse Kinematics using Jacobian methods: https://homes.cs.washington.edu/~todorov/courses/cseP590/06_JacobianMethods.pdf

Report

Please answer the following questions in your report:

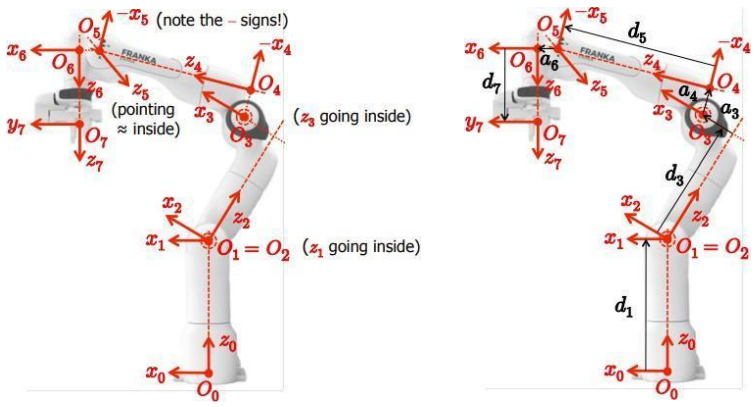
1. About task 1 (15%)

1.1 Briefly explain how you implement **your_fk()** function **(3%)**

- (You can paste the screenshot of your code and explain it)

1.2 What is the difference between D-H convention and Craig's convention (Modified D-H Convention)? **(2%)**

1.3 Complete the D-H table in your report following **D-H convention (10%)**



The coordinate frames of the robot arm in this homework following D-H convention

i	d	α (rad)	a	θ_i (rad)
1				θ_1
2				θ_2
3				θ_3
4				θ_4
5				θ_5
6				θ_6
7				θ_7

A D-H table example format (please fill in it in your report)

2. About task 2 (15% + 5% bonus)

2.1 Briefly explain how you implement **your_ik()** function **(10%)**

- (You can paste the screenshot of your code and explain it)

2.2 What problems do you encounter and how do you deal with them? **(5%)**

2.3 Bonus! Do you also implement other IK methods instead of pseudo-inverse methods? How about the results? **(5% bonus)**

Grading Policy

1. **Correctness verification using test cases 70%**
    - task 1: 30%
    - task 2: 40%
2. **Report 30% + bonus 5%**
    - about **task 1**: 15%
    - about **task 2**: 15% + 5% bonus

QA page

The following link is to the QA page for Homework 4. If you have any questions, please post your questions to this Notion page. We will answer them as soon as possible.

Link To Notion Page

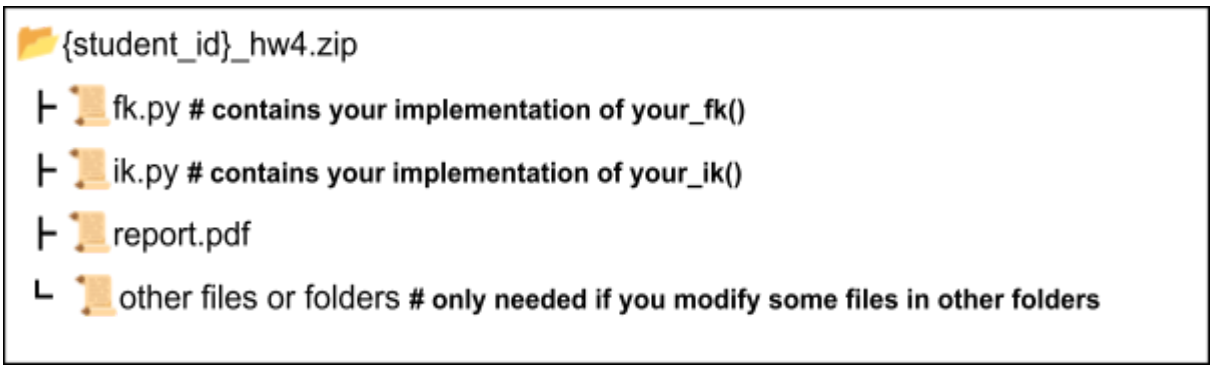
Submission

Due Date: 2026/5/12 23:59

Please directly compress your code files and report (.pdf) into

{STUDENT_ID}_hw4.zip and submit it to the New E3 System.

The file structure should look like:



- **Wrong submission format leads to -10 points.**
- **Late submission leads to -20 points per day.**

- **Plagiarism is strictly prohibited, including assignments written on GitHub. If we find any instances, it will result in a zero score without any room for objection.**
- **Reports not written in English will receive a score of zero.**